KMAA

ÉDITION 2026.10

PRODUIT • TECHNIQUE • LIVRAISON

KMAA
E-TICKETS

Cahier de pilotage

État des lieux du produit, modules présents, paramètres observés et trajectoire vers une mise en service fiable.

PORTEUR	BASE D'ANALYSE	DATE
Converge SAS	Dépôt Flutter fourni	21 octobre 2026

Estimation indicative en jours-personne. Les services backend et les comptes prestataires ne sont pas inclus dans le dépôt audité.

01 / VUE DE DIRECTION

Synthèse exécutive

Le produit existe côté application ; le système complet n’est pas démontré.

Le dépôt contient une application Flutter multiplateforme et plusieurs parcours métier. Il appelle une API externe : son code, son contrat publié, les prestataires de paiement et l’infrastructure ne sont pas présents ici.

69 fichiers Dart

Inventaire source : 1 point d’entrée et 68 fichiers sous mon-app.

3.41.6 / Dart 3.11.4

Versions observées dans l’environnement d’analyse.

Tests bloqués

Le test fourni référence MyApp, absent du code actuel.

Chaîne fonctionnelle observée

APP FLUTTER — API REST / JSON — SERVEUR EXTERNE

Lecture immédiate

- Parcours déjà présents dans le client : compte, recherche de trajet, panier/récapitulatif, billets, notifications, profil, favoris, rappels et avis.
- Écarts critiques : paiement explicitement simulé ; OTP/social non branchés en production ; contrôle agent sur données locales ; pas de suite de tests valide.
- Priorité de pilotage : sécuriser l’achat et sa validation côté serveur avant d’ajouter des fonctionnalités de confort.

Convention de statut dans ce document

« Présent » signifie que le code client a été trouvé. Cela ne prouve pas le fonctionnement de bout en bout, la conformité du backend ni la recette sur appareils.

02 / PORTEFEUILLE PRODUIT

Fonctionnalités : état constaté

Domaine	État dans le dépôt	Preuve / nuance de gestion
Compte, connexion, OTP	Partiel	Écrans et appels API présents. OTP en mode démo documenté ; fournisseurs sociaux en messages de débogage.
Recherche et réseau	Présent / à valider	Compagnies, villes, routes, horaires, favoris. Données locales inégales ; source serveur attendue.
Réservation de trajet	Présent côté client	Sélection passagers/âge, date et horaire ; contrat et capacité serveur non vérifiés.
Paiement	Simulé	Écran attend 1,4 s et simule l'étape de paiement ; intégration LigdiCash indiquée comme à faire.
Billets / QR / PDF	Présent côté client	Liste, détail, QR, export/partage PDF, cache Hive et synchronisation. Validité du QR à confirmer serveur.
Report de billet	Présent côté client	Lecture de créneaux et soumission du report exposées via API ; règles métier à recetter.
Présence à la gare	Présent / à sécuriser	Coordonnées facultatives ; permission iOS configurée dans Info.plist au moment de l'analyse.
Contrôle agent gare	À renforcer	Recherche et vérification dans les billets locaux ; contrôle serveur non observé dans l'écran.
Notifications et rappels	Partiel / à valider	Cache et file d'actions hors ligne, rappels locaux, voix et vibration ; fonctionnement plateforme non testé.
Profil, avis, support	Partiel	Modification profil et avis API présents ; horaires/contact du support incomplets.

Ce qui peut être annoncé prudemment

L'application propose des parcours de billetterie et des services associés. Ne pas annoncer comme « opérationnels » le débit réel, l'envoi WhatsApp, l'authentification sociale ou la validation anti-fraude tant que les preuves serveur et recette ne sont pas produites.

03 / CARTE DES MODULES

Architecture fonctionnelle du client

Le projet est organisé par domaines d'écrans Flutter, avec une couche API centralisée, des stores locaux Hive et des utilitaires transverses. La navigation et les états ne sont pas encore documentés dans un guide produit dédié.

A. Accès & identité

Accueil / bienvenue
Inscription et validation
Connexion
Mot de passe oublié
Profil et déconnexion

B. Découverte & achat

Compagnies et favoris
Villes et réseaux
Trajets nationaux / externes
Passagers par âge
Récapitulatif et paiement

C. Après-vente

Mes billets / détails
QR et reçu PDF
Report de voyage
Avis de gare
Notifications et rappels

D. Terrain & support

Vérification agent gare
Signalement de présence
Localisation / cartes
Appel au service client
Alertes vocales et vibration

Socle technique transversal

Composant	Responsabilité constatée
ApiService	Appels HTTP, configuration base URL, erreurs ApiException, session et jeton.
Stores Hive	Billets, utilisateur, notifications, favoris et avis disponibles localement.
Navigation	MaterialApp, routes déclarées, route dynamique de détail compagnie.
Services natifs	Notifications locales, fuseaux horaires, TTS, vibration, géolocalisation, liens/composeur.

Frontière d'analyse

Backend, base de données serveur, contrats OpenAPI, traitements asynchrones prestataires, comptes WhatsApp et paiement réel : absents du dépôt fourni, donc non audités.

04 / PARAMÈTRES & DÉPENDANCES

Configuration à maîtriser

Paramètres techniques observés

Paramètre	Valeur observée	Action de pilotage
API_URL	Dart define ; défaut localhost:8000/api, Android émulateur 10.0.2.2.	Imposer HTTPS en production ; séparer dev / recette / prod.
Délai API	20 s ; restauration session /me : 6 s.	Confirmer stratégie retry et comportement hors ligne.
Session	Clé secure storage : kmaa_token.	Valider expiration / révocation côté serveur et tests reprise.
Android	Java/Kotlin 17 ; app ID com.example.kmaa_e_tickets ; release signé debug.	Définir identifiant commercial et signature release.
iOS localisation	NSLocationWhenInUseUsageDescription présent dans Info.plist.	Tester acceptation/refus et absence de permission.
Données & notifications	Hive, notifications locales, timezone, TTS, vibration.	Matrice de recette Android / iOS / Web à formaliser.
Web	README décrit serveur web et API_URL ; métadonnées web encore génériques.	CORS, domaine, HTTPS, identité PWA et mode release.

Dépendances directes principales

Flutter/Dart, http, flutter_secure_storage, Hive/Hive Flutter, geolocator, flutter_local_notifications, timezone, flutter_tts, vibration, video_player, qr_flutter, pdf/printing, url_launcher et crypto. Versions et contraintes sont déclarées dans pubspec.yaml / pubspec.lock ; SDK Dart demandé : ^3.11.4.

Décision de mise en production

La configuration par défaut utilise HTTP et une URL locale. Tant qu'un endpoint HTTPS par environnement, une configuration CORS Web et des secrets côté serveur ne sont pas confirmés, considérer la configuration comme développement uniquement.

Opérateurs cités par l'interface : Visa, Coris, Telecel, Moov, Orange ; le paiement externe est annoté comme une intégration à remplacer par LigdiCash. Présence d'une option UI ≠ connectivité acquéreur.

05 / PLAN DE LIVRAISON ESTIMATIF

Backlog pour atteindre le niveau production

Estimations en jours-personne (j.p.), affinées après accès au backend, contrats prestataires et critères de recette. Elles couvrent analyse, développement et validation de la tâche indiquée, pas la disponibilité d'un tiers.

Prio	Lot / résultat attendu	Est.	Critère de sortie
P0	Remettre en état la suite de tests et CI (constructeur ConvergeApp, tests smoke)	2–4 j.p.	flutter test passe ; pipeline reproductible.
P0	Paiement réel sécurisé (client + API + callback / idempotence)	6–12 j.p.	Un achat autorisé, refusé, annulé et rejoué est traçable sans double débit.
P0	Validation agent côté serveur (QR signé, anti-rejeu, droits agent)	4–7 j.p.	Billet validé une fois ; statut vérifié en ligne ; cas expiré/refusé testés.
P0	OTP de production et suppression du mode démo (WhatsApp/SMS, limites, récupération)	3–6 j.p.	Aucun code affiché en UI ; délivrance et throttling testés.
P1	Recette E2E réservation / achat / report + contrats API	4–7 j.p.	Parcours nominal et erreurs métier couverts sur recette.
P1	Stabiliser API, erreurs et environnements (HTTPS, secrets, logs, timeouts)	3–5 j.p.	Dev/recette/prod séparés ; erreurs observables ; aucun secret client.
P1	Matrice plateforme localisation / notifs (Android, iOS, Web)	2–4 j.p.	Permissions, refus, redémarrage, timezone et rappels vérifiés.
P1	Préparation publication et exploitation (signature, identifiants, monitoring, support)	3–6 j.p.	Build signé, fiche store/web, crash reporting et procédure support.
P2	Compléter données transport et support (tarifs, réseau, horaires/contact)	2–4 j.p.	Données métier approuvées ; états vides documentés.

TOTAL INDICATIF : 29–55 J.P. Soit environ 6–11 semaines pour une personne, hors attente prestataire et accès serveur.

Les lots P0 sont des conditions de confiance et de sécurité, non des améliorations facultatives. Certains lots peuvent avancer en parallèle si backend, produit et opérations sont disponibles.

06 / SÉQUENCEMENT PROPOSÉ

Trajectoire vers une version maîtrisée

Phase	Séquence	Sortie attendue
0 · Cadrage (2–3 j.p.)	Obtenir dépôt backend, contrat API, comptes sandbox paiement/WhatsApp, règles de validation et acteurs responsables.	Périmètre serveur clarifié, parcours et critères de recette signés.
1 · Fiabilisation (2–4 j.p.)	Réparer test de base, ajouter CI, fixer environnements, rendre visibles les erreurs de démarrage.	Socle vérifiable à chaque changement.
2 · Transactions (9–18 j.p.)	Construire paiement réel avec backend ; validation agent autoritaire ; OTP production.	Achat et contrôle billet sécurisés et auditables.
3 · Recette (6–11 j.p.)	Tests de bout en bout, report/notifications, permissions, tests appareils et données limites.	PV de recette métier et techniques, anomalies bloquantes closes.
4 · Publication (3–6 j.p.)	Signing, identifiants, confidentialité, monitoring, support et publication progressive.	Version candidate, plan de retour arrière et suivi de production.

Dépendances qui pilotent le calendrier

- Paiement : accès sandbox et documentation opérateur + endpoints serveur/webhook avant estimation ferme.
- OTP : fournisseur, modèle de message, règles d’expiration et quotas convenus avec backend.
- Contrôle agent : identité / rôles agents, format de QR et stratégie de connexion hors ligne décidés.
- Publication : propriété des comptes Apple/Google, identifiant d’application et certificats disponibles.

Risques à traiter en conception

Double paiement, QR falsifié/rejoué, billet local périmé, partage de position non consenti, API en HTTP, panne réseau et notifications non exécutées par certains systèmes.

Décision demandée à l’équipe

Définir le mode de contrôle gare (connexion obligatoire ou mécanisme offline signé) et les statuts de remboursement/annulation avant d’implémenter la passerelle réelle.

07 / QUALITÉ, PREUVES & GOUVERNANCE

Conditions de clôture et limites

Résultats d’exécution dans l’environnement fourni

Contrôle	Résultat	Implication
flutter --version	Flutter 3.41.6 ; Dart 3.11.4	Environnement cohérent avec la contrainte SDK déclarée.
flutter test	ÉCHEC : test/widget_test.dart instancie MyApp, classe absente.	Aucune preuve de tests automatisés valides actuellement.
flutter analyze	ÉCHEC : 44 constats, dont 1 erreur de compilation du test.	Aussi warnings de variables / symboles inutilisés et infos dépréciation / conventions.
Recette sur appareil / serveur	Non exécutée dans cette analyse.	Aucune preuve de production, paiement, WhatsApp ni notifications natives.

Definition of Done recommandée

- Tests Flutter et analyse statique sans erreur bloquante ; pipeline CI exécutable sur chaque PR.
- Tests intégration simulant succès, refus, timeout, doublons, réponse invalide et session expirée.
- Recette signée du parcours recherche → achat → émission billet → contrôle agent → report / notification.
- Matrice Android/iOS/Web sur permissions, réseau intermittent, timezone, reprise après fermeture.
- Configuration production HTTPS, clés de signature, politique de confidentialité et procédure incident approuvées.

Limites et confiance des estimations

Audit statique du dépôt Flutter fourni, fichiers de build et tests disponibles. Le backend, les données en production, les identifiants secrets, la disponibilité des prestataires, la conformité juridique et la performance n’ont pas été vérifiés. Le planning 29–55 j.p. est un ordre de grandeur à recalibrer après le cadrage.

Prochaine action

Désigner un responsable API/paiement et ouvrir l’accès sandbox.

Gouvernance

Revue hebdomadaire des lots, preuves de recette et risques ouverts.

Identité, Firebase et accès sociaux

Exigences de réalisation pour compléter les parcours d'inscription et de connexion de KMAA E-Tickets.

> **État constaté :** l'application présente des boutons Google, Apple et Facebook, mais le dépôt ne contient pas de dépendances Firebase ni de connexion effective à ces fournisseurs. Le parcours actuellement raccordé côté client est l'authentification par téléphone et mot de passe, avec OTP WhatsApp.

Décision d'architecture recommandée

Conserver l'API métier comme source de vérité des comptes, des billets et des autorisations. Firebase Authentication sert à authentifier les personnes avec Google ou Apple ; le client transmet ensuite le jeton Firebase à l'API KMAA. Le serveur valide ce jeton auprès de Firebase Admin, associe l'identité au compte KMAA et renvoie le jeton de session propre à l'API. Un jeton Firebase ne remplace pas le jeton API existant.

Parcours	Comportement attendu	État / contrat à prévoir
Téléphone	Inscription, validation du code, connexion par téléphone/mot de passe.	Routes clientes déjà appelées ; envoi réel WhatsApp et règles serveur à valider.
Google	Connexion via Google, récupération du jeton Firebase, échange contre une session KMAA.	À développer ; fournisseur Google activé dans Firebase Console.
Apple	Connexion via « Sign in with Apple », échange du jeton Firebase contre une session KMAA.	À développer ; fournisseur et capacités Apple configurés.
Mot de passe oublié	Pour les comptes téléphone : code WhatsApp, nouveau mot de passe, puis ouverture d'une session.	Ce flux est déjà appelé côté client. Il ne dépend pas de Firebase.
Récupération par e-mail	À proposer uniquement si la connexion par e-mail/mot de passe devient un mode de compte supporté.	À décider ; utiliser alors le flux Firebase Auth de réinitialisation e-mail.

Configuration Firebase et plateformes

1. Créer des projets Firebase séparés pour développement, recette et production ; activer Authentication et les fournisseurs Google et Apple.
2. Enregistrer les identifiants Android (applicationId) et iOS (Bundle ID) réellement utilisés ; générer la configuration par environnement avec FlutterFire CLI.
3. Google : renseigner les empreintes de signature Android SHA-1/SHA-256, les clients OAuth et les URL de redirection requises pour les plateformes ciblées.
4. Apple : activer la capacité « Sign in with Apple » dans l'identifiant d'application, créer et configurer Service ID / Team ID / clé si requis, puis vérifier le retour d'autorisation sur iOS.
5. Ajouter et initialiser les paquets nécessaires (notamment `firebase_core`, `firebase_auth` et les connecteurs Google/Apple retenus), puis tester chaque plateforme réelle.

Contrat serveur et comptes existants

Ajouter un échange explicite, par exemple `POST /auth/firebase`, recevant le jeton Firebase et le fournisseur. Le serveur vérifie signature, audience, émetteur, expiration et état du compte ; il ne fait jamais confiance au profil envoyé par le mobile. La réponse suit le format de session de l'API (`token`, `user`). Les comptes déjà liés au même numéro ne sont associés qu'après preuve de possession ; les comptes ne sont pas fusionnés uniquement parce que deux fournisseurs retournent la même adresse e-mail.

Les identifiants et secrets de service Firebase appartiennent au serveur / coffre de secrets. Aucun secret Admin, clé privée Apple ou secret OAuth confidentiel ne doit être embarqué dans Flutter, commité ou affiché dans les journaux.

À confirmer avant développement : stratégie de liaison des comptes, plateformes incluses, prise en charge d'une connexion e-mail et règles de désactivation d'un compte.

WhatsApp OTP : configuration de bout en bout

Le client appelle déjà les routes d'inscription, de vérification et de renvoi OTP, ainsi que le parcours de réinitialisation. Le dépôt ne contient pas le backend ni les identifiants du prestataire : l'acheminement WhatsApp réel reste donc à mettre en place et à tester.

Configuration du prestataire

1. Choisir et contractualiser le compte WhatsApp Business Platform (API Cloud Meta ou prestataire agréé), vérifier le numéro d'envoi, le compte Business et les règles locales d'utilisation.
2. Créer et faire approuver les modèles WhatsApp de catégorie authentification (inscription, renvoi, récupération), avec code à usage unique, durée d'expiration et langue(s) supportée(s).
3. Configurer côté serveur l'identifiant du compte, le numéro d'expéditeur, le jeton d'accès et la version d'API ; stocker les secrets dans un coffre, séparément par environnement.
4. Configurer le webhook de statut de livraison (et sa vérification), les alertes de panne, le suivi des quotas/coûts et une procédure de renouvellement du jeton.
5. Exécuter les essais de recette avec des numéros autorisés, puis avec un numéro client réel après validation des modèles et du consentement requis.

> **Ne pas confondre :** l'écran « WhatsApp » et les appels `/auth/*` ne prouvent pas que WhatsApp Business est configuré. Le code OTP est généré, vérifié, expiré et invalidé par le backend ; Flutter ne fabrique ni ne valide le code.

Règles de sécurité OTP à faire appliquer côté serveur

- Code à usage unique, lié au numéro et au motif (inscription ou récupération), stocké sous forme non réutilisable et invalidé après validation ou remplacement.
- Durée de vie, nombre d'essais, délai de renvoi et plafonds par numéro, adresse IP et appareil définis puis validés en recette. Cible initiale proposée : expiration en 5 minutes, au plus 5 essais par code et 60 secondes avant renvoi.
- Limitation des abus, réponses génériques pour ne pas révéler si un numéro possède un compte, et contrôles de fraude avant d'émettre des messages payants.
- Ne jamais enregistrer le code OTP, le mot de passe, les jetons d'accès ou le contenu complet des messages dans les journaux ou outils d'analytique.
- Messages d'interface compréhensibles en cas de délai, échec de livraison, code erroné/expiré ou limite de renvoi ; permettre un nouvel essai sans créer de compte ou billet en double.

Les valeurs de durée et de quotas ci-dessus sont des propositions de recette à ratifier avec le responsable produit, le backend et les règles du fournisseur WhatsApp.

Contrat API d'authentification observé

Méthode et route	Rôle / entrée attendue	Résultat attendu
`POST /auth/register`	Prénom, nom, téléphone, mot de passe ; déclenche l'OTP.	Compte en attente de vérification ; réponse métier exploitable.
`POST /auth/verify-otp`	Téléphone et code à 6 chiffres.	Session API et utilisateur après validation.
`POST /auth/resend-otp`	Téléphone ; limites de renvoi appliquées par le serveur.	Confirmation sans divulgation d'existence du compte.
`POST /auth/login`	Téléphone et mot de passe.	Jeton de session API et profil minimal.
`POST /auth/forgot-password`	Téléphone ; envoi d'un OTP de récupération.	Réponse identique pour numéro connu/inconnu.
`POST /auth/reset-password`	Téléphone, code, nouveau mot de passe.	Code invalidé, mot de passe remplacé, session si confirmé par le contrat.
`POST /auth/logout`	Jeton API courant.	Révocation serveur si prise en charge et suppression locale du jeton.

Le client possède un délai réseau général de 20 s (6 s pour restaurer la session). Documenter les délais fournisseur, erreurs transitoires et règles de reprise côté backend.

API locale : configuration développeur

Le backend n'est pas présent dans le dépôt Flutter audité. Il doit être démarré séparément ; `API_URL` désigne sa racine API, suffixe `/api` inclus.

Client Flutter	URL à utiliser pour l'API locale	Remarque
Navigateur sur le même PC	`http://localhost:8000/api`	Valeur par défaut Web et desktop. Le backend doit autoriser l'origine Web via CORS.
Émulateur Android	`http://10.0.2.2:8000/api`	Alias de l'hôte depuis l'émulateur Android ; valeur par défaut du client.
Téléphone physique	`http://IP_LAN_DU_PC:8000/api`	Téléphone et PC sur le même réseau ; écouter sur l'interface réseau, autoriser le port au pare-feu.
Test partagé / tunnel	`https://hote-public/api`	Utiliser l'URL HTTPS publique du backend et autoriser l'origine du tunnel dans CORS.

Commandes de lancement :

- Émulateur Android : `flutter run --dart-define=API_URL=http://10.0.2.2:8000/api`
- Téléphone : `flutter run --dart-define=API_URL=http://192.168.1.20:8000/api` (remplacer par l'adresse IPv4 du PC).
- Web : `flutter run -d web-server --web-hostname 0.0.0.0 --web-port 8080 --dart-define=API_URL=http://localhost:8000/api`

> **Attention Web :** si un collègue ouvre l'application depuis un autre ordinateur, son `localhost` pointe vers son propre poste. Passer à l'URL HTTPS du tunnel/backend. Ne jamais utiliser HTTP non chiffré en production.

Routes métier appelées par le client Flutter

Domaine	Routes observées
Compte	`GET /me`, `POST /me/profil`, échange Firebase à créer : `POST /auth/firebase`.
Catalogue	`GET /compagnies`, `GET /compagnies/{slug}/villes-depart?type=...`, `GET /compagnies/{slug}/routes?depart=...&type=...&date=...`.
Billets	`POST /billets`, `GET /billets`, `GET /billets/{reference}/creneaux?date=...`, `POST /billets/{reference}/reporter`, `POST /billets/{reference}/presence`.
Notifications	`GET /notifications`, `POST /notifications/{id}/lue`, `POST /notifications/lire-tout`, `DELETE /notifications/{id}`.
Autres	`POST /avis-gares` ; routes d'authentification listées à la page précédente.

Pré-requis de lancement local et contrat

- Fournir le dépôt backend, sa version de runtime, sa commande de démarrage, ses variables d'environnement d'exemple sans secret, sa base locale/migrations et des données de démonstration.
- Publier un contrat OpenAPI décrivant schémas, champs obligatoires, authentification, statuts HTTP et erreurs. Le client s'attend notamment à recevoir `token`/`user` après authentification et un objet `billet` après achat/report.
- Réserver `401` à une session invalide, `409` à un conflit de disponibilité et `422` à une validation ou un montant erroné ; normaliser le JSON d'erreur (`message`, code, erreurs de champ).
- Configurer CORS pour les seules origines Web autorisées ; ne pas exposer le service de base de données, clés de paiement, secrets WhatsApp ou secrets Firebase Admin au client.

Les routes listées sont celles appelées dans le code Flutter inspecté ; leur disponibilité, leur comportement et leur schéma doivent être confirmés dans le backend et le contrat OpenAPI.

Comportements attendus avant livraison

Les écrans présents ne suffisent pas à démontrer un parcours métier complet. Chaque exigence doit être testée contre le backend et les prestataires réels ou leurs environnements de recette.

ID	Exigence fonctionnelle	Critère d'acceptation vérifiable
F-01	Inscription et vérification téléphonique	Un numéro valide reçoit un OTP WhatsApp ; code correct, incorrect, expiré, consommé ou renvoyé produit le résultat attendu sans doublon de compte.
F-02	Connexion, session et déconnexion	Connexion valide ouvre une session ; redémarrage restaure une session valide ; jeton refusé efface la session ; déconnexion supprime le jeton local et invalide le serveur si prévu.
F-03	Récupération du mot de passe	Demande par téléphone envoie un OTP sans révéler l'existence du compte ; code valide permet un nouveau mot de passe et l'ancien devient inutilisable.
F-04	Connexion Google et Apple	Après consentement fournisseur, le backend vérifie le jeton Firebase, retrouve/crée le compte selon les règles approuvées, et renvoie une session KMAA ; annulation et compte bloqué sont gérés proprement.
F-05	Recherche de trajet	Compagnies, départs, destinations, horaires, prix et places proviennent du serveur ; filtres par date/type ne présentent pas de places périmées comme garanties.
F-06	Achat et paiement	Le serveur recalcule prix, taxes et disponibilité ; confirmation de paiement vérifiée côté serveur, traçable et idempotente ; refus/annulation ne crée aucun billet payé.
F-07	Billet, QR et report	Le billet confirmé est consultable et partageable ; QR vérifiable côté serveur ; règles de report, échéance, tarif et capacité sont contrôlées atomiquement.
F-08	Notifications et présence	Lecture, marquage et suppression suivent l'état serveur ; la localisation reste facultative, demandée au bon moment et non nécessaire pour utiliser le billet.
F-09	Profil, consentement et erreurs	Le client peut corriger les informations permises, voir le consentement requis, reprendre après erreur réseau et recevoir un message compréhensible sans fuite de données sensibles.

Scénarios minimum de recette

- OTP livré et non livré ; mauvais numéro ; code erroné/expiré ; limite de tentatives ; renvoi temporisé ; mot de passe oublié sur compte existant et numéro inconnu.
- Connexion Google/Apple réussie, consentement annulé, première connexion, retour d'un utilisateur, jeton Firebase expiré, compte KMAA déjà lié et utilisateur désactivé.
- API inaccessible, lente, répondant 401/409/422/429/5xx ; reprise réseau ; restauration et déconnexion de session.
- Recherche sans résultat ; places épuisées entre sélection et achat ; montant recalculé différent ; paiement approuvé, refusé, annulé ou callback reçu deux fois.
- Vérification du billet et du QR, changement de date, notifications sur plusieurs appareils, permissions de localisation refusées et fonctionnement en réseau intermittent.

> **Preuve de sortie :** associer chaque scénario à un résultat attendu, une capture/log expurgé, un environnement, une version et un responsable de validation. Les tests de paiement et d'envoi WhatsApp doivent utiliser des comptes de recette avant activation de production.

À faire signer par produit, backend, mobile, exploitation et responsable des paiements avant le démarrage des lots d'intégration.

Exigences non fonctionnelles et critères de mise en service

Les seuils ci-dessous sont des cibles initiales de recette à confirmer selon la charge prévue, les contrats fournisseurs et les obligations réglementaires applicables.

Qualité	Exigence / mesure de réception proposée
Sécurité	HTTPS obligatoire hors boucle locale ; validation côté serveur de chaque ressource ; mots de passe hachés avec algorithme adaptatif ; OTP à usage unique ; jetons révocables/expirables ; protection contre brute force, énumération, rejeu et injection.
Données personnelles	Collecter uniquement les données utiles ; documenter finalités, consentement, rétention, suppression/export et droits d'accès ; chiffrer les sauvegardes et les flux ; ne pas journaliser secrets, OTP, jetons ou données de paiement.
Performance	Cible initiale : 95 % des lectures API courantes en moins de 2,5 s sous la charge de recette convenue ; achats et paiement soumis à des délais explicités, sans double transaction.
Disponibilité / reprise	Définir disponibilité cible et fenêtre de maintenance ; sauvegarde/restauration vérifiées ; détection des erreurs et procédure de reprise documentées ; aucune promesse de fonctionnement hors ligne pour une opération qui exige une validation serveur.
Fiabilité	Opérations d'achat, paiement, génération de billet et callbacks idempotents ; retries bornés uniquement pour opérations sûres ; erreurs réseau distinguées des refus métier.
Compatibilité	Valider les versions minimales Android/iOS, appareils réels, tailles d'écran et navigateurs Web supportés ; vérifier les capacités biométriques/fournisseurs sur plateformes cibles.
Accessibilité	Libellés accessibles, contraste suffisant, taille de texte adaptable, navigation clavier/lecteur d'écran et messages d'erreur compréhensibles.
Observabilité	Logs structurés avec identifiant de corrélation ; métriques de latence, erreurs, OTP et paiement ; alertes opérationnelles ; données sensibles expurgées.
Maintenabilité	Contrat OpenAPI versionné, environnements séparés, configuration documentée, CI reproductible et tests automatisés bloquants avant fusion/livraison.

Environnements et gestion des secrets

- Développement, recette et production séparés : API, Firebase, WhatsApp, paiements, bases et comptes de test distincts.
- Fournir un fichier de variables d'exemple sans valeurs sensibles ; injecter les secrets depuis l'environnement serveur / coffre de secrets. Ne pas mettre de secret dans `--dart-define`, le dépôt ou l'application compilée.
- En production, imposer une URL HTTPS, domaines CORS restreints, certificats valides, règles de sauvegarde et clés de paiement de production soumises à autorisation.
- Tracer la rotation/révocation des clés Firebase Admin, WhatsApp, paiement et signature ; définir les responsables et la procédure d'incident.

Gates avant ouverture au public

1. Contrat API et matrice d'erreurs validés ; backend déployable localement et en recette à partir d'instructions reproductibles.
2. Tests unitaires, widget, intégration et contrats passent dans la CI ; tests sur Android/iOS et smoke tests Web ciblent les plateformes réellement annoncées.
3. Google/Apple testés avec configurations de production ; WhatsApp Business vérifié et modèles approuvés ; paiements testés avec callbacks signés et rejoués.
4. Audit des permissions, flux de données personnelles, sécurité du QR, règles de rétention, sauvegardes et restauration terminé ; aucune clé de test dans l'application publique.
5. Recette métier signée, tableaux de bord/alertes activés, plan de support et procédure de rollback disponibles.

> **Décisions à consigner :** pays et contraintes de données applicables, seuils de performance/disponibilité, durée OTP et quotas, modes d'authentification retenus, politique de liaison des comptes, plateformes supportées, prestataires WhatsApp/paiement et propriétaire du backend.

Complément établi à partir du dépôt Flutter fourni et des appels API observés. Il décrit des exigences de réalisation, pas une preuve que le backend ou les prestataires sont déjà configurés.